WEEK-05

Java Application Connectivity with MySQL Database using JDBC in NetBeans

Aim

To create a Java application in NetBeans that establishes a connection with a MySQL database using JDBC and displays the records.

Software Requirements

- Windows 10/11 Operating System
- Java Development Kit (JDK)
- Apache NetBeans IDE
- WAMP Server
- MySQL Database
- MySQL JDBC Connector (Connector/J)

Algorithm

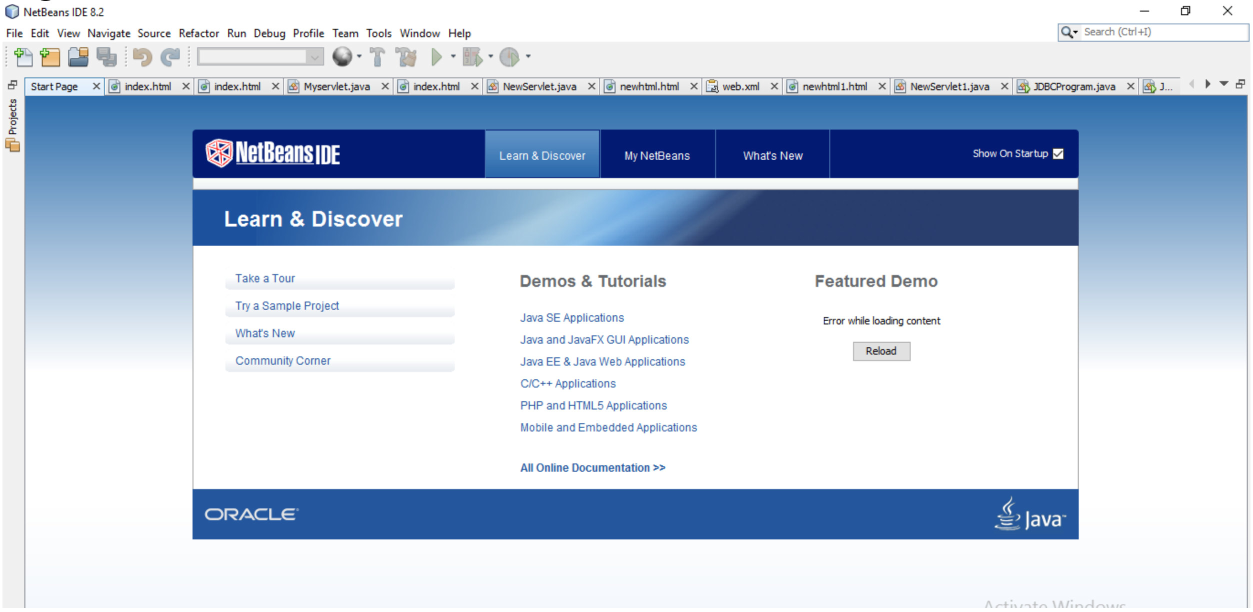
1. Launch the NetBeans IDE.
2. Create a new Java Application project.
3. Start the WAMP Server and ensure MySQL is running.
4. Create a database and the required employee table in MySQL.
5. Insert sample employee records into the table.
6. Add the MySQL JDBC driver to the project's Libraries folder.
7. Write the JDBC program to establish a database connection.
8. Compile and execute the Java application.
9. Verify that the employee details are displayed successfully.

Procedure

Step 1

Open **Apache NetBeans IDE**.

Figure 1: NetBeans IDE Home Screen



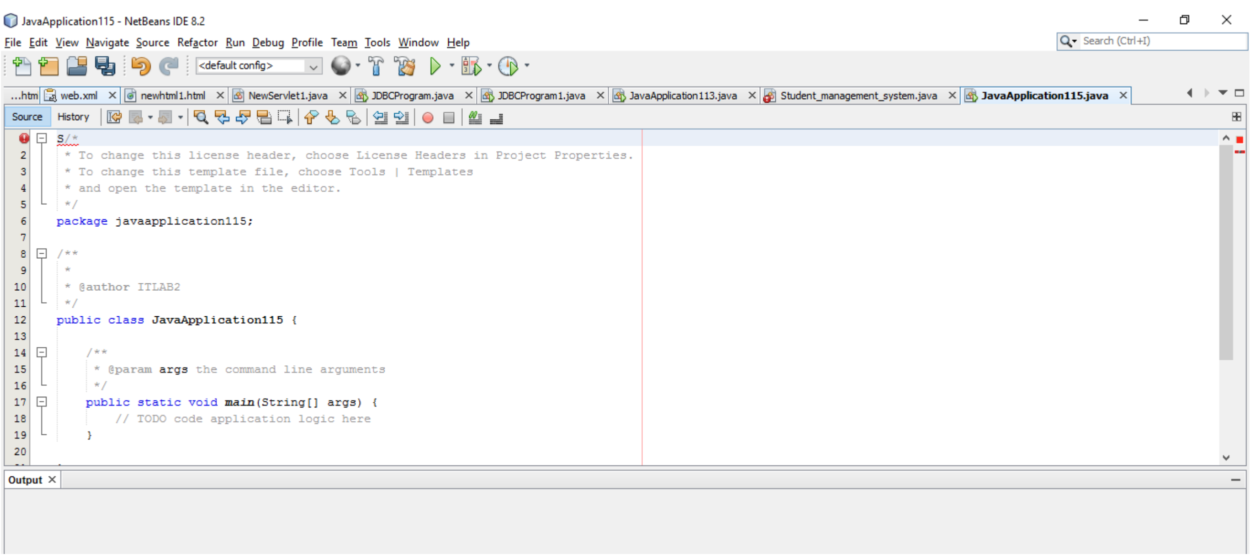
Step 2

Go to:

File → New Project → Java → Java Application

Enter the project name as **JavaApplication26**, then click **Finish**.

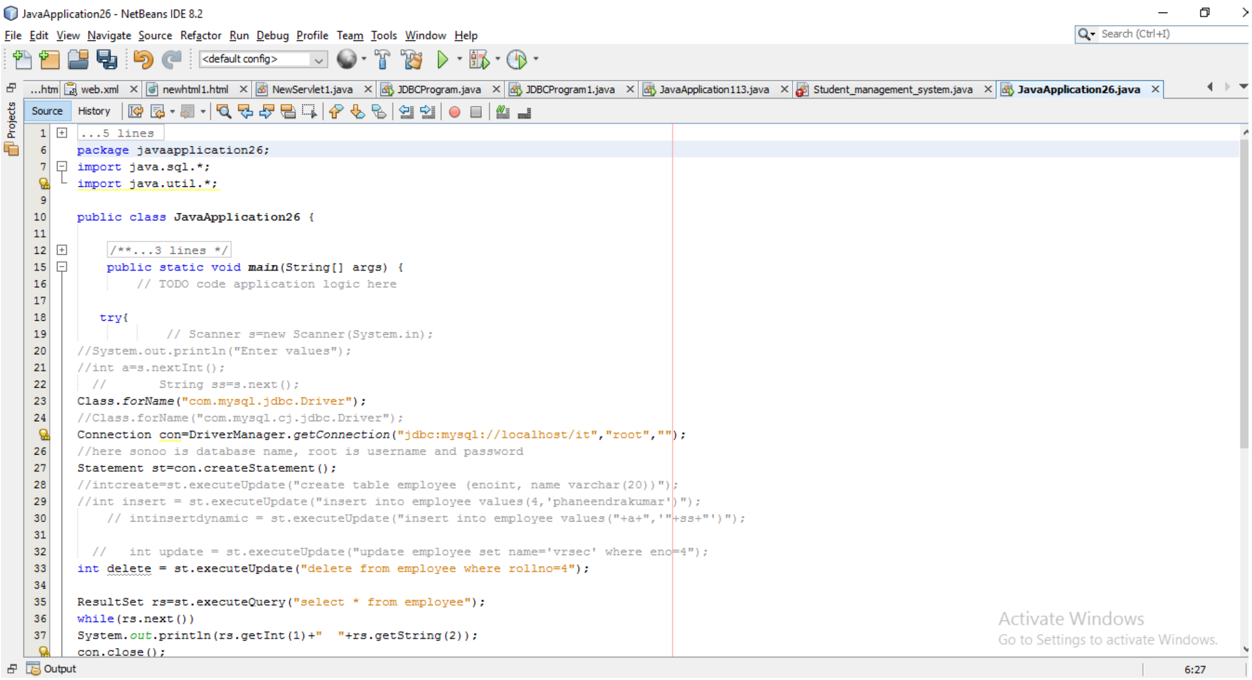
☐ **Figure 2:** Creating a New Java Project



Step 3

Paste the provided Java source code into the editor.

☐ **Screenshot 3:** Java Source Code in NetBeans



Step 4

Start **WAMP Server**.

Open MySQL Console.

Execute the following SQL commands:

```
create database it;

use it;

create table employee(
rollno varchar(10),
name varchar(100) primary key
);

insert into employee values(1,'ramesh');
insert into employee values(2,'harshith');
insert into employee values(3,'haswika');
insert into employee values(4,'rajesh');
insert into employee values(5,'suresh');
```

☐ **Screenshot 4:** Database Creation

```
mysql> CREATE DATABASE it;
Query OK, 1 row affected (0.02 sec)
```

☐ **Screenshot 5:** Employee Table Created

```
mysql> use it;
Database changed
mysql> create table employee(rollno varchar(10),name varchar(100) primary key);
Query OK, 0 rows affected (0.02 sec)
```

☐

Screenshot 6: Records Inserted Successfully

```
ysql> insert into employee values(1,'ramesh');insert into employee values(2,'harshith');insert into employee values(3,'haswika');insert into employee values(4,'rajes
h');insert into employee values(5,'suresh');
Query OK, 1 row affected (0.01 sec)

Query OK, 1 row affected (0.00 sec)

Query OK, 1 row affected (0.00 sec)

Query OK, 1 row affected (0.00 sec)

Query OK, 1 row affected (0.00 sec)
```

Step 5

In NetBeans

Project → JavaApplication26

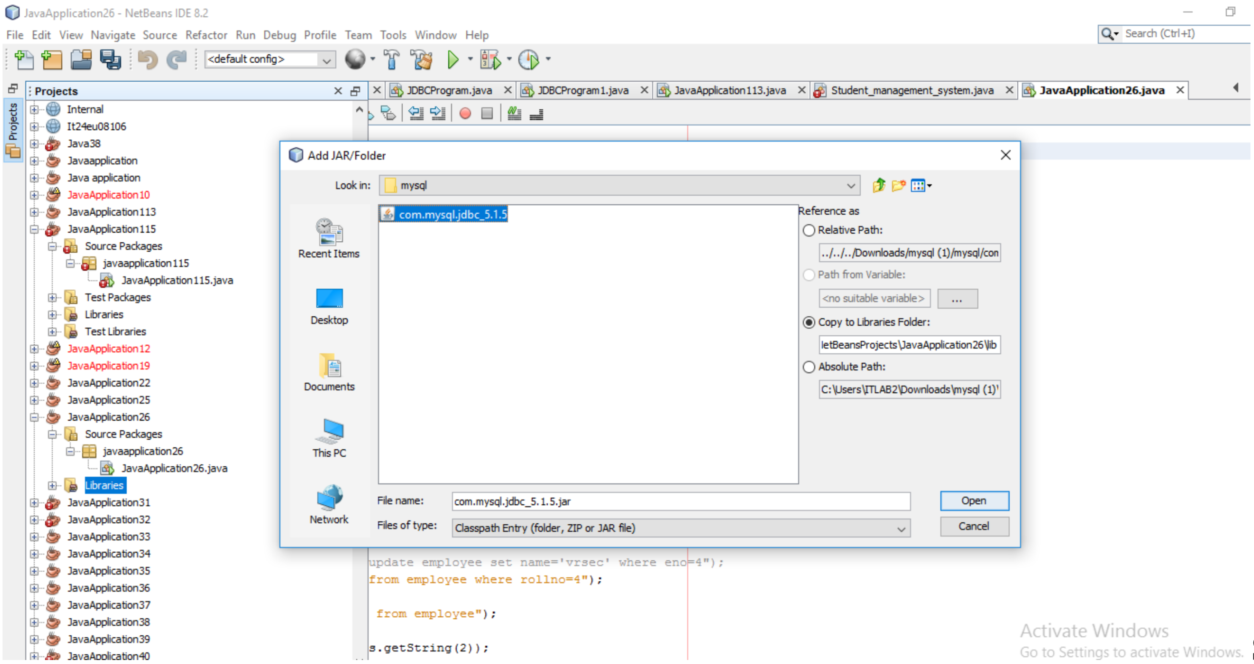
Right Click

Libraries

Select

Add JAR/Folder

Choose the MySQL JDBC Driver.

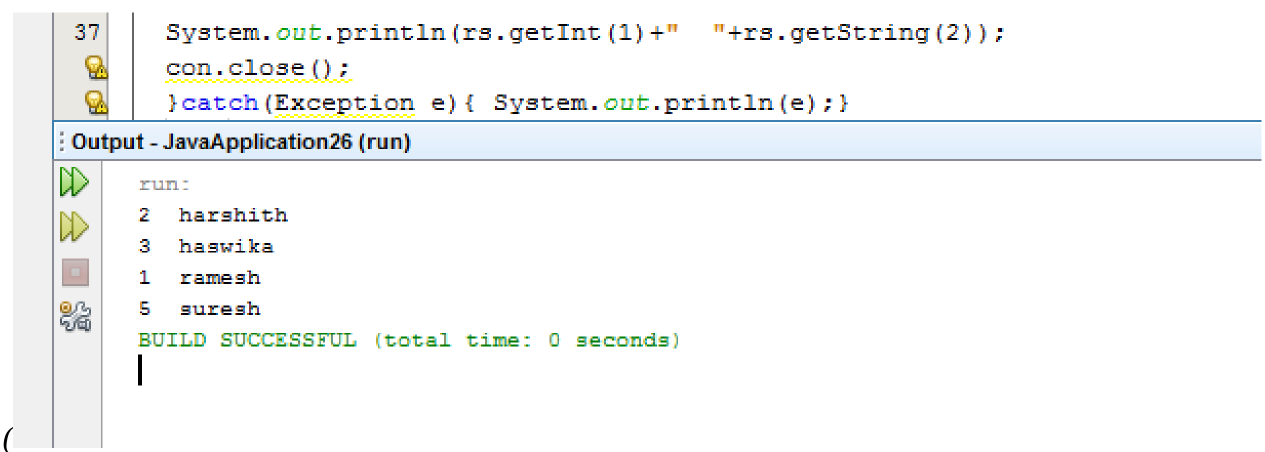


Step 6

Click the **Run** button.

The Java program connects to MySQL and displays employee records.

☐ **Screenshot 8:** Program Execution



(

Java Program

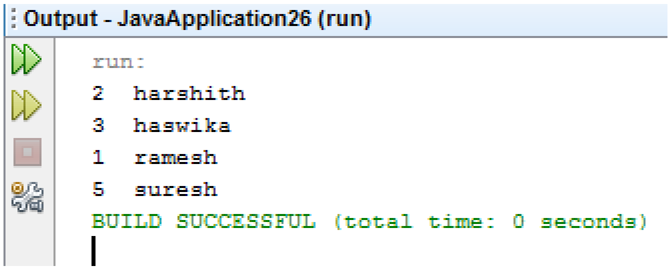
/*
* To change this license header, choose License Headers in Project Properties.
* To change this template file, choose Tools | Templates
* and open the template in the editor.
*/
package javaapplication26;

```
import java.sql.*;
import java.util.*;

public class JavaApplication26 {

/**
* @param args the command line arguments
*/
public static void main(String[] args) {
// TODO code application logic here

try{
// Scanner s=new Scanner(System.in);
//System.out.println("Enter values");
//int a=s.nextInt();
//      String ss=s.next();
Class.forName("com.mysql.jdbc.Driver");
//Class.forName("com.mysql.cj.jdbc.Driver");
Connection con=DriverManager.getConnection("jdbc:mysql://localhost/it","root","");
//here sonoo is database name, root is username and password
Statement st=con.createStatement();
//intcreate=st.executeUpdate("create table employee (enoint, name varchar(20))");
//int insert = st.executeUpdate("insert into employee values(4,'phaneendrakumar')");
// intinsertdynamic = st.executeUpdate("insert into employee values("+a+",'"+ss+"')");

//   int update = st.executeUpdate("update employee set name='vrsec' where eno=4");
int delete = st.executeUpdate("delete from employee where rollno=4");

ResultSet rs=st.executeQuery("select * from employee");
while(rs.next())
System.out.println(rs.getInt(1)+"  "+rs.getString(2));
con.close();
}catch(Exception e){ System.out.println(e);}
}

}
```

Expected Output

```
Roll No     Name

1           ramesh
2           harshith
3           haswika
4           rajesh
5           suresh
```

□ **Screenshot 9:** Final Output Window



Result

The Java application was successfully connected to the MySQL database using JDBC. The records stored in the **employee** table were retrieved and displayed successfully.